Chapter 3: Lexical Analysis

The Compiler's Front Line

By
Dr. Adnan M. Sherif, PhD

CS432 – Spring 2025/2026

1

What is Lexical Analysis

What is the first thing you do when you read a sentence in a new language?

Response: You separate it into individual words.

This is the function of Lexical Analysis.

2

Where Are We in the Compiler?

- Lexical Analyzer is the first step of the Compilation process.
- Input: Stream of characters.
- Output: Stream of tokens.
- Why separate it?
  - Simplicity: Removes clutter (comments, whitespace).
  - Efficiency: Specialized techniques are faster.
  - Portability: Device-specific input handled once.

3

Key Concepts: Tokens, Patterns, Lexemes

- **Token:** A syntactic category (e.g., identifier, number, if, while).
- **Pattern:** A rule (often a regex) defining the set of strings for a token.
  - identifier: [a-zA-Z_][a-zA-Z0-9_]*
  - number: [0-9]+
- **Lexeme:** The actual character sequence matched (count, 123, if).
- **Analogy:**
  - **Token = Word type** (e.g., Noun, Verb)
  - **Pattern = Definition** of that word type
  - **Lexeme = The actual word** (e.g., "compiler", "run")

4

Handling Errors & Other Tasks

- **Errors:** What if the lexer sees $?
  - It must report an error. It can't just crash.
  - Common strategy: skip the invalid character and continue.
- **Other Tasks:**
  - Removing whitespace and comments.
  - Expanding macros (in some languages).
  - Recording line numbers for error messages.

5

We Need a Formal Language to Describe Patterns

- How do we *precisely* tell the lexer what an identifier looks like?

We use Regular Expressions.

6

The Alphabet of Regex

- **Basic Operations:**
  - **Alternation (|)**: r|s - a string from language r **or** language s.
    - Example: a|b matches "a" or "b".
  - **Concatenation (**rs**)**: a string from r followed by a string from s.
    - Example: (a|b)(c|d) matches "ac", "ad", "bc", "bd".
  - **Kleene Closure (**r*****)**: zero or more strings from r.
    - Example: (a|b)* matches "", "a", "b", "aa", "ab", "ba", "bb", "aaa", ...
- **Shorthands:**
  - r+ = rr* (one or more)
  - r? = (r|ε) (zero or one)
  - [a-z] = a|b|c|...|z (character class)
  - . = any character (except newline, usually)
  - "a" = literal character a (to escape operators)

7

Lexical Ambiguity and Resolution

- **Problem:** What if a lexeme matches multiple patterns?
  - if matches the pattern for both keyword and identifier.
  - >= is a prefix of >>= (in some languages).
- **Solution: Two Rules:**
  - **Longest Match:** The lexer should choose the pattern that matches the longest string. >>= is taken as one token, not two > and =.
  - **Rule Priority:** For two patterns matching the *same length*, the pattern listed **first** in the lexer specification wins. The keyword rule for if must come before the identifier rule.

8

Regex is a Specification, We Need an Implementation

- How does the lexer *check* if a string matches a regex?

It uses a Finite Automaton (FA).

9

How Lex/Flex Works (The Magic)

- **Input:** A .l file containing regex patterns and actions.
- **Output:** A C program (lex.yy.c) containing a DFA-based lexer.
- **Internal Steps:**
  - **Regex -> NFA:** Using Thompson's Construction. Each regex operator builds a small NFA. Big NFAs are built by combining small ones.
  - **NFA -> DFA:** Using the **Subset Construction** algorithm. The states of the DFA are *sets of states* from the NFA. This eliminates the "guessing" (nondeterminism).
  - **DFA Minimization:** Hopcroft's algorithm minimizes the number of DFA states, making the resulting lexer smaller and faster.

10

The Final Lexer Code

- The generated lex.yy.c code is essentially a **transition table** representing the DFA, coupled with a driver loop.
- The lexer has to read one character past the end of a token to know it's done, and then "back up" the input.

```
c
state = START_STATE;
lexeme = "";
while (state != ERROR_STATE) {
  c = get_next_char();
  lexeme += c;
  state = transition_table[state][c];
  if (state is ACCEPTING_STATE) {
    backup_input_to_end_of_lexeme(); // Crucial for longest match!
    return token_for[state];
  }
}
report_error();
```

11

Buffering Input

- **Problem:** Reading one character at a time from disk is *slow*.
- **Solution: Use a buffer!**
- **The Two-Buffer Scheme (Buffer Pairs):**
  - Use two halves of a large array (e.g., 4096 bytes total).
  - While the lexer is reading from one half, the OS is refilling the other half with the next chunk of data from the file.
  - This hides I/O latency and is incredibly efficient.

12

The Sentinel Trick

- **Problem:** The check if (buffer is empty) at every character is expensive.
- **Solution:** Place a **sentinel** character (like EOF) at the end of the valid data in the buffer. The test for end-of-buffer becomes a test for this special character, which can be combined with the check for the current character.
  - if (current_char == EOF) -> handle buffer refill.
  - This is a classic, low-level optimization.

13

Writing a Lexer by Hand vs. Using Lex/Flex

- **Use Lex/Flex:**
  - Faster development.
  - Fewer errors.
  - Easier maintenance.
- **Write by Hand:**
  - For *extremely* simple languages.
  - When you need maximum, absolute control over performance.
  - When you need to integrate with a very unusual environment.

14

Exercise: Design a Lexer Spec

- **Task:** In groups, write a small Lex/Flex specification for a tiny language that has:
  - Keywords: sky, blue
  - Integers
  - Identifiers
  - Operators: +, ++
  - Whitespace
- **Crucially:** They must order their rules correctly to handle the + vs ++ longest match rule and the sky vs identifier priority rule.

15

Sample lex file

```
%%
"sky"     { return KEYWORD_SKY; }
"blue"    { return KEYWORD_BLUE; }
"++"      { return OP_INCR; }
"+"       { return OP_PLUS; }
[0-9]+    { return NUMBER; }
[a-zA-Z]+  { return IDENTIFIER; }
[ \t\n]   { /* skip */ }
.         { printf("Unexpected char: %s\n", yytext); }
%%
```

16

Handling comments

- Comments in the program code have no use to the compiler and should be skipped.
- Single line comment:

  // if(x==0) { return 1}
- Note that the above example should be ignored by the Lexer and should not generate any tokens.
- To achieve this, we include a rule to ignore comments and simple increment the line number.

  “//”.\n {++yylineno;}

17

Handling block comments

- Block comments stretch over several lines. In C, block comments are marked with “*” to indicate the beginning of the comment block and “*/” to indicate the end of comment block.
- All text and lines between the two marks should be ignored by the lexer and not associated to tokens.
- In this case we use states to indicate that a special treatment is need.
- Lex/Flex use INITIAL state for the constructs of the language.
- We use the BEGIN(State name) to start a new state, the rules for the new state are separated from the language rules in a block “{}” with the name of the state in the beginning of the block.

18

Handling block comments - Continue

```
lex
%%
"/*" { BEGIN(COMMENT); } // Start of a comment, enter COMMENT state

<COMMENT>{
  "*/"   { BEGIN(INITIAL); } // End of comment, return to initial state
  \n    { ++yylineno; }    // Count newlines inside comments
  .     { /* ignore all other characters */ }
  <<EOF>> {
    fprintf(stderr, "Error: Unterminated comment at line %d\n", yylineno);
    yyterminate(); // Or handle the error gracefully
  }
}
```

19

Handling strings

- Defining a regular expression for a string literal delimited by " in Lex (or Flex) requires careful handling of escape sequences and the closing quote.
- A naive approach would be:

  \"[^"]*\"
- This means:
  - \" - a double quote
  - [^"]* - zero or more of any character that is *not* a double quote
  - \" - a closing double quote
- This pattern breaks if the string itself contains a double quote, even if it's escaped (e.g., "She said, \"Hello\""). The [^"] part will match the backslash \, and the next character " will be seen as the closing quote, leaving the final " unmatched.

20

Handling strings

- This is the cleanest method, as it allows you to handle the string content in a separate state, making it easy to catch an unterminated string.

```
%x STRING /* Define an exclusive start condition called STRING */
#define STRING_TOKEN 1001
%%
\"    { BEGIN(STRING); } /* On opening ", enter the STRING state */
<STRING>{
 \"    { BEGIN(INITIAL);
         return STRING_TOKEN;
       } /* On closing ", return to initial state */
 \n    { /* Often you might want to handle newlines, maybe as an error */ }
 \\[\\"nrt] { /* Match common escape sequences: \\, \", \n, \r, \t */ }
 .     { /* This will match any other character inside the string */ }
 <<EOF>> {
    fprintf(stderr, "Error: Unterminated string at line %d\n", yylineno);
    yyterminate();
 }
}
```

21